

PROYECTO

bot_olt

Bot de Telegram para consulta de datos OLT

Estado	Desarrollo funcional completo. Probado contra la base real.
Pendiente	18 items para pasar a produccion (6 bloqueantes)
Alcance	4 consultas de lectura, exportacion a Excel/CSV/TXT, login con clave propia, reseteo de contraseñas del portal y auditoria completa
Convencion	README_PYTHON_BOT.md del repositorio estructura (GERET)
Fecha	1 de septiembre de 2026

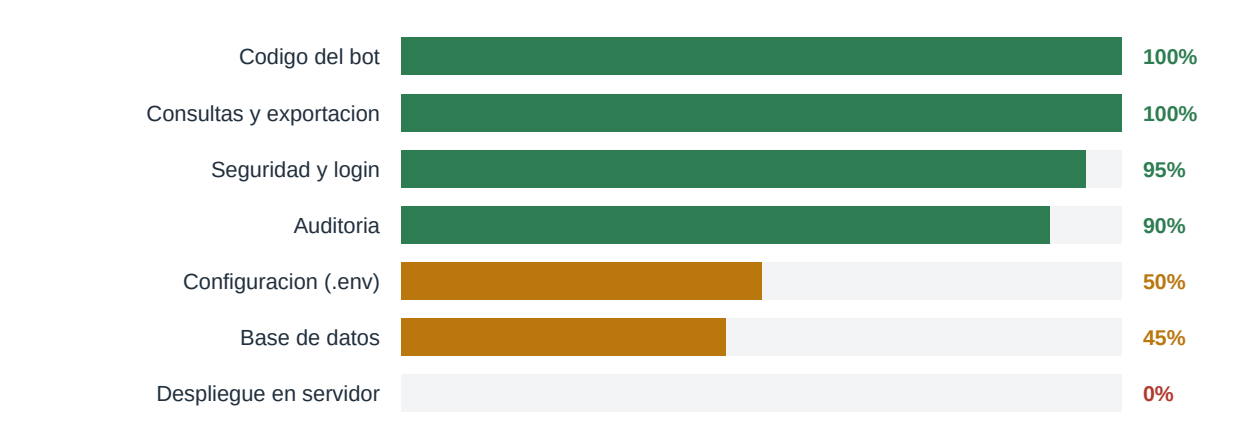
1. Resumen ejecutivo

El bot consulta datos de OLT sobre el esquema **Aden** y entrega los resultados en el chat, con descarga en Excel, CSV o TXT. Incorpora ademas el reseteo de contraseñas de usuarios del portal, que hasta ahora solo se hacia desde la web, y deja registro auditable de cada accion.

El acceso tiene dos capas: el chat de Telegram debe estar registrado, y la persona debe autenticarse con una **clave propia del bot**, independiente de la del portal. Asi, la clave del portal nunca viaja por Telegram.

El codigo esta completo y verificado contra la base real: las cuatro consultas devuelven datos y la exportacion funciona en los tres formatos. Lo que falta para produccion no es desarrollo, sino **configuracion, ejecucion de dos scripts SQL y validacion de reglas de negocio** que no pudieron confirmarse desde el codigo existente.

Avance por area



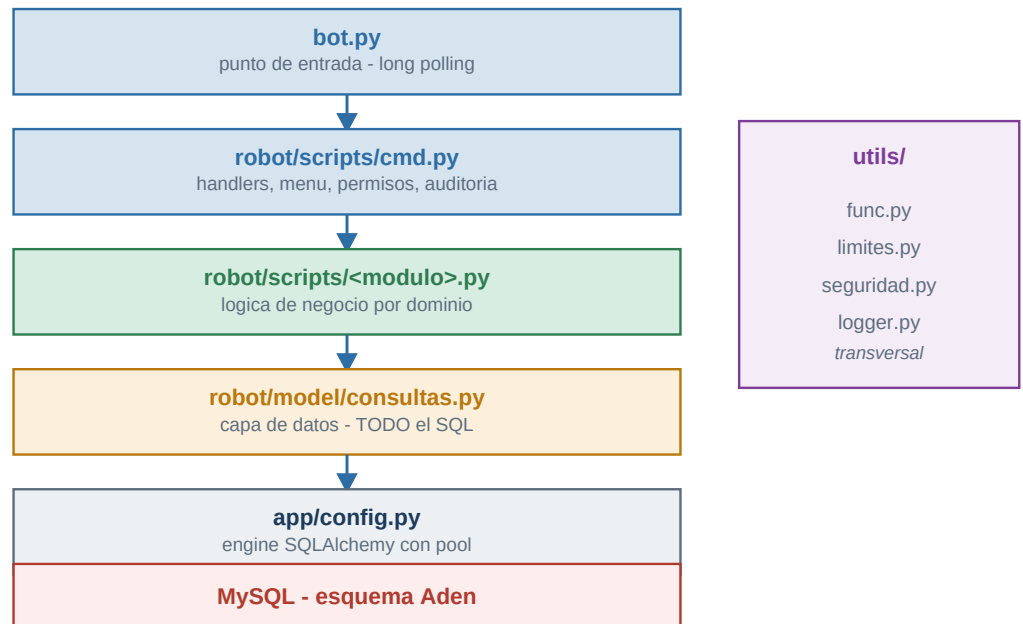
Las areas en rojo dependen de datos o accesos que estan fuera del codigo.

Cifras del proyecto

Metrica	Valor	Metrica	Valor
Archivos Python	17	Consultas de lectura	4
Lineas totales	3.252	Operaciones de escritura	8
Tablas nuevas	2	Formatos de exportacion	3
Pruebas automatizadas	55 (todas pasan)	Tablas consultadas	8

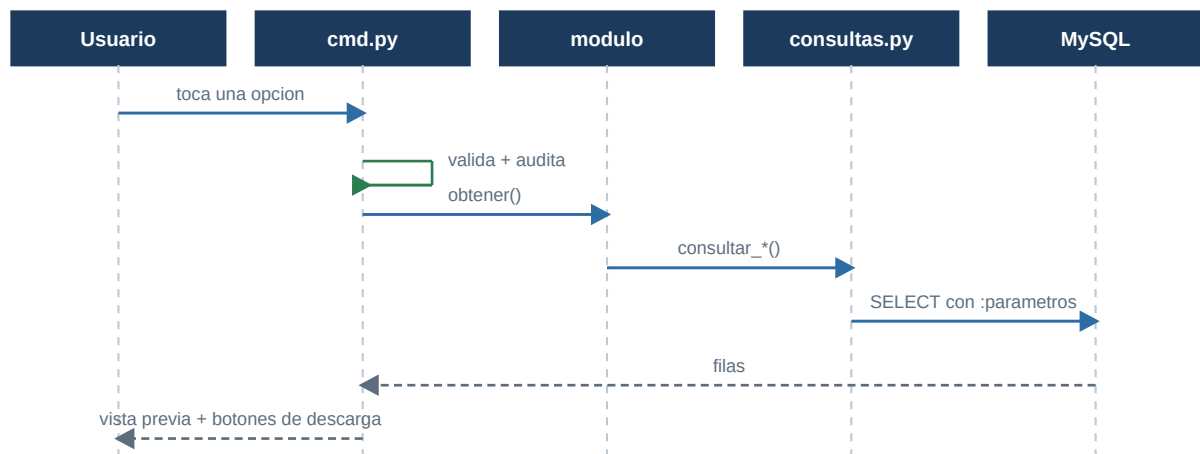
2. Arquitectura

La estructura sigue las capas que define **README_PYTHON_BOT.md**. Cada capa conoce solo a la siguiente: el SQL vive unicamente en `consultas.py` y las conexiones solo en `app/config.py`. Esa separacion tuvo un beneficio concreto durante el desarrollo: los dos errores que aparecieron al probar estaban cada uno en su capa y se corrigieron sin tocar el resto.



Flujo de dependencias entre capas. `utils/` es transversal y no contiene logica de negocio ni SQL.

Flujo de una consulta



Las funciones de datos son async y ejecutan el SQL en un thread aparte (`asyncio.to_thread`), para no bloquear el bot mientras corre la consulta.

3. Funcionalidades implementadas

Consultas disponibles en el menu

Opcion	Tablas	Parametro
Inventario OLT	OLT_SERVER	ninguno
Puertas PON	OLT_SERVER, OLT_ONT_PCS, OLT_TRAFIGOPON, OLT_PUERTOS_UPLINKS	fecha (obligatoria)
Alarmas criticas	OLT_ALARMA_CRITICAL_LOS	ninguno
Trafico PON	OLT_TRAFIGOPON	IP de la OLT
Cambiar mi clave	OLT_BOT_USUARIOS (escritura)	clave actual y nueva
Cerrar sesion	OLT_BOT_USUARIOS (escritura)	ninguno
Resetear contraseña	OLT_USUARIOS (escritura)	usuario - solo perfil 1
Asignar clave del bot	OLT_BOT_USUARIOS (escritura)	usuario - solo perfil 1

Puertas PON exige fecha porque OLT_TRAFIGOPON es una tabla muy grande y la consulta debe acotarse siempre.

Como se agrega una consulta nueva

Tres pasos, un archivo por capa: la query en consultas.py, el modulo de negocio en robot/scripts/, y una linea en el diccionario OPCIONES. El teclado, el routing, la exportacion a los tres formatos y la auditoria funcionan sin escribir nada adicional.

Reseteo de contraseña

Replica el case "changePassw" del portal: deja la clave en 123456, actualiza last_connection y activa al usuario. Pide el nombre exacto, avisa si no existe, cancela si hay mas de una coincidencia, y muestra usuario y estado antes de pedir confirmacion. Mejora tres cosas respecto del portal: usa parametros ligados en vez de concatenar el SQL, restringe por perfil, y deja registro de quien reseteo a quien.

4. Seguridad y proteccion de la base

El bot es publico: cualquiera que conozca su nombre de usuario en Telegram puede escribirle. Como consulta una base de **produccion**, cada mensaje pasa por cuatro filtros ordenados del mas barato al mas caro, de modo que un mensaje no autorizado se descarte antes de llegar a la base.



El caso critico es la etapa 3: se cachea tambien el resultado negativo. Sin eso, un no autorizado generaria una consulta por cada mensaje enviado.

Login con clave propia del bot

La cuarta etapa es una capa de autenticacion sobre el registro por chat_id: tener el telefono de alguien autorizado ya no basta. La clave es **independiente de la del portal**, de modo que la del portal nunca viaja por Telegram y, si esta se filtra, la cuenta del portal no se ve afectada. Por lo mismo no usa MD5 sino PBKDF2-SHA256 con salt por usuario, solo con la libreria estandar de Python.



Primer ingreso de un usuario nuevo. El mensaje que contiene la clave se borra del chat apenas se procesa.

La sesion y el contador de intentos fallidos viven en la base, no en memoria: reiniciar el bot no reabre sesiones ni reinicia los contadores. Tras MAX_INTENTOS_LOGIN fallos la cuenta se bloquea unos minutos, y el permiso se revalida sin cache antes de cada escritura privilegiada.

Controles implementados

Riesgo	Estado	Como se resuelve
Inyeccion SQL	Cubierto	Parametros ligados en todas las queries. Verificado: no hay interpolacion de texto del usuario en ningun SQL.

Riesgo	Estado	Como se resuelve
Entradas invalidas	Cubierto	Validadores de fecha, IP y texto en utils/func.py. El dato invalido nunca llega a la base.
Abuso / fuerza bruta	Cubierto	Limite por chat_id en ventana deslizante, previo a la consulta de autorizacion.
Consultas por no autorizados	Cubierto	Cache de autorizacion (incluye el negativo) y silenciado tras 3 intentos.
Acceso no autorizado	Cubierto	Tabla OLT_BOT_USUARIOS por chat_id; operaciones privilegiadas ademas por perfil del portal.
Telefono robado o desatendido	Cubierto	Login con clave propia del bot y sesion con vencimiento. Cambiar la clave exige la actual.
Fuerza bruta sobre la clave	Cubierto	Bloqueo de la cuenta tras N intentos, con el contador en la base y no en memoria.
Clave del bot en la base	Cubierto	PBKDF2-SHA256 con salt por usuario, solo libreria estandar. El mensaje con la clave se borra del chat.
Secretos en los logs	Corregido	loguru se configuro con diagnose=False. Antes volcaba el token y la cadena de conexion en cada traceback.
Agotamiento del pool	Pendiente	pool_timeout sigue en 300s y no hay timeout de query. Ajuste postergado por decision del responsable.
Hash del portal (reseteo)	Deuda	El reseteo de la clave del PORTAL sigue usando MD5 por compatibilidad obligada con su login. No afecta a la clave del bot.

5. Auditoria

Cada evento queda en la tabla **OLT_BOT_AUDITORIA** con quien, que y cuando. El registro no esta dentro de cada handler sino en el punto donde se despachan las opciones del menu, de modo que **una opcion nueva queda auditada sin escribir una linea extra**. Son seis puntos de registro en total.

Accion	Cuando se registra
INICIO_SESION	El usuario abre el bot con /start o /menu
CONSULTA	Se elige una opcion del menu, y de nuevo al enviar su parametro
DESCARGA	Se descarga un archivo Excel, CSV o TXT
RESETEO_PASSWORD	Se confirma un reseteo; guarda a que usuario se le aplico
ACCESO_DENEGADO	Escribe un chat_id no autorizado (acotado por el silenciado)
LOGIN_OK / LOGIN_FALLIDO	Resultado de cada intento de autenticacion
CUENTA_BLOQUEADA	Se agotaron los intentos permitidos
LOGOUT	El usuario cierra su sesion
CAMBIO_CLAVE_BOT	El usuario cambia su propia clave
ASIGNACION_CLAVE_BOT	Un admin asigna la clave inicial a alguien

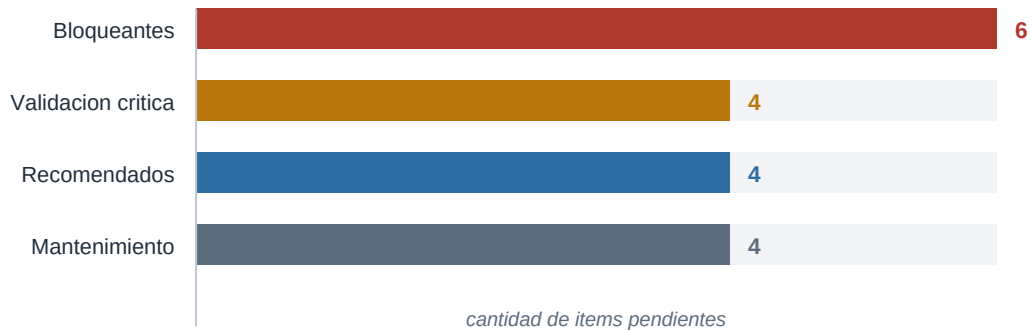
Ejemplo de registro

fecha	usuario	accion	detalle	parametro
19:40:12	jperez	INICIO_SESION	/start	
19:40:20	jperez	CONSULTA	Puertas PON	
19:40:31	jperez	CONSULTA	puertas_fecha	2026-08-10
19:40:45	jperez	DESCARGA	Puertas PON	excel
19:41:02	jperez	RESETEO_PASSWO RD	OK	mgonzalez

El archivo BD/OLT_BOT_AUDITORIA.sql incluye cuatro consultas de auditoria listas para usar y la sentencia de purga documentada.

6. Pendientes para pasar a produccion

Quedan **18 items**. Ninguno es desarrollo pendiente: son configuraciones, tres scripts SQL por ejecutar y reglas de negocio por confirmar. Los seis primeros son bloqueantes: sin ellos el bot no opera correctamente en produccion.



Bloqueantes

#	Pendiente	Por que bloquea
1	Revocar y regenerar el BOT_TOKEN con /revoke, actualizar el .env y borrar log/bot.log	El token quedo expuesto en un traceback. Quien lo tenga controla el bot por completo.
2	Ejecutar BD/OLT_BOT_AUDITORIA.sql y dar permiso INSERT sobre la tabla	Sin la tabla no hay auditoria: los eventos solo caen al log de archivo, que rota cada 7 dias.
3	Ejecutar BD/OLT_BOT_USUARIOS_v2.sql, reemplazando antes 'tu_usuario' por el administrador real en el PASO 2	Agrega la clave del bot y la sesion. Sin el PASO 2 nadie puede entrar: todas las claves las asigna un admin desde el bot y no habria ninguno habilitado.
4	Dar de alta los usuarios en OLT_BOT_USUARIOS (la tabla existe pero esta vacia)	El bot rechaza a todos. Cada persona obtiene su chat_id escribiendole al bot.
5	Instalar Python 3.9 a 3.12 en el servidor	3.6 no soporta asyncio.to_thread; 3.13+ no tiene wheels para las versiones fijadas.
6	Completar ecosystem.config.js con la ruta real de despliegue	PM2 no levanta el bot con las rutas de ejemplo (/var/www/bot_olt).

Validacion critica

El bot funciona, pero estos puntos determinan si los datos que entrega son correctos. Conviene resolverlos antes de que la gente confie en los numeros.

#	Pendiente	Riesgo si no se valida
7	Confirmar las tres reglas de negocio de Puertas PON: que un puerto utilizado es up_mbps mayor a 1; que valores de marca son uplink; y que zs_comercial es el total de puertas	El porcentaje de ocupacion sale incorrecto. El PHP original tiene un error (marca = '2' OR '1' siempre es verdadero) que se interpreto como marca IN (1,2), sin poder confirmarlo.
8	Probar el reseteo con un usuario de prueba y verificar que puede entrar al portal con 123456	Si el hash MD5 no coincide con lo que espera el login, se deja al usuario sin acceso.
9	Confirmar si OLT_USUARIOS.usuario tiene indice UNIQUE	Sin UNIQUE, la proteccion contra resetear varias cuentas depende solo del chequeo en codigo.
10	Decidir si un usuario dado de baja en el portal (OLT_USUARIOS.estado = 0) debe perder el acceso al bot	Hoy el bot solo valida OLT_BOT_USUARIOS.activo: quien sale del portal conserva el acceso al bot mientras no se le desactive tambien alli. No se agrego por no haber sido pedido.

Recomendados antes de produccion

#	Pendiente	Detalle
11	Bajar pool_timeout de 300s a ~30s y agregar timeout de query	Postergado por decision del responsable. Hoy, si el pool se agota, cada peticion espera 5 minutos.
12	Verificar la salida a api.telegram.org desde el servidor	Si la red sale por proxy, hay que completar PROXY en el .env o el bot no conecta.
13	Decidir la migracion del hash MD5 del portal	Solo afecta al reseteo de claves del portal. La clave del bot ya usa PBKDF2.
14	Decidir si las alarmas se envian automaticamente por horario	Hoy solo responden a demanda. El envio programado requiere tarea periodica y suscriptores.

Mantenimiento y documentacion

#	Pendiente	Detalle
15	Commitear bot_olt al repositorio Pruebas	El proyecto esta sin versionar. Tampoco esta versionada la skill geret-estructura.
16	Sincronizar los archivos entre los dos PCs de trabajo	El desarrollo ocurrio en un equipo y las pruebas contra la base en otro.
17	Documentar el rango de Python soportado (3.9 a 3.12) en README_PYTHON_BOT.md	Cambio de convencion: se propone en el repositorio estructura, no se aplica por cuenta propia.
18	Definir la politica de purga de OLT_BOT_AUDITORIA	Se decidio no purgar por ahora. La sentencia esta documentada en el .sql.

Orden sugerido de ejecucion

Paso	Accion	Verificacion
1	Revocar el token e instalar Python 3.12 en el servidor	El token viejo responde 401
2	Ejecutar los tres scripts de BD/ y verificar permisos	python -m tests.test_conexion
3	Completar el .env: token, conexion, PERFILES_ADMIN=1	El bot arranca sin advertencias en el log
4	Dar de alta al primer administrador y habilitarlo en el PASO 2 del script v2	Ingresa con 123456 y el bot le exige cambiarla
5	Dar de alta al resto desde el bot, con 'Asignar clave del bot'	Cada uno entra y define su clave
6	Validar Puertas PON contra datos conocidos de la red	El porcentaje de ocupacion cuadra
7	Probar el reseteo del portal con un usuario de prueba	Entra al portal con 123456
8	Desplegar con PM2	pm2 logs bot_olt sin errores

7. Incidencias resueltas durante el desarrollo

Se documentan porque varias son trampas que volveran a aparecer al montar el entorno en otro equipo o en el servidor.

Sintoma	Causa real	Solucion
pip fallaba al instalar pandas pidiendo un compilador de C	El PC tenia Python 3.14, sin wheels para pandas 2.1.4; pip intentaba compilar numpy	Instalar Python 3.12 y mantener las versiones que fija la convencion
python --version seguia mostrando 3.14 tras crear el venv	El entorno virtual no estaba activado	Activar con env\Scripts\activate antes de verificar
Puertas PON no respondia nada	SQL invalido: se uso MAX() dentro del WHERE de una subconsulta	Reescribir con tabla derivada; ademas recorre la tabla una vez y no una por OLT
TypeError al restar puertas	zs_comercial es varchar: MySQL devuelve texto, no numero	Helper a_entero() antes de operar
El token aparecio escrito en el log	loguru trae diagnose=True y volcaba el valor de cada variable del stack	Configurar diagnose=False conservando el stack
El menu quedaba atrapado pidiendo una fecha	Tocar otra opcion se interpretaba como respuesta a la pregunta pendiente	Dar prioridad al menu sobre la entrada pendiente

Hallazgo heredado del portal

En Desarrollo/puertas pon/option_gpon.php la exclusion de uplinks esta escrita como WHERE marca = '2' OR '1'. En MySQL eso se evalua como (marca = '2') OR ('1'), y '1' siempre es verdadero: el subquery devuelve todos los puertos y el NOT IN los excluye todos. Es un error preexistente en el portal, no introducido por el bot, y conviene revisarlo tambien alli.